

Lab 4, Multiple Regression and Inference with Real Data

Estimation, Regression Tables, t, F and LM Tests, Logs, and Diagnostic Plots

2026-10-02

Contents

Introduction	2
Packages and data	2
Estimating a multiple regression	3
A regression table	4
Storing estimates and using them in text	5
The t test	6
Two tailed test	6
One tailed test	7
Testing a value other than zero	8
The F test	8
By hand	9
By command	9
Overall significance of the regression	10
The LM test	10
By hand	10
By command	11
Creating log variables	11
Graphs after a regression	14
Coefficient plot	14
Actual against fitted values	15
Distribution of the residuals	16
The built in diagnostic plots	18
Plots to detect heteroskedasticity	19
Residuals against fitted values	19
Squared residuals against fitted values	20
Residuals against each regressor	21
Scale location plot	22
Confirming the picture with a test	23
Summary and questions to think about	24

Introduction

In Labs 2 and 3 we worked with a population we built ourselves, so we always knew the true slope. Real research is different. We get one sample, we never see the truth, and everything we say about the population has to come from that one sample. This lab moves to real data and walks through the full routine of an applied regression exercise, from estimation to a results table to hypothesis tests to diagnostic plots.

We use the `hprice1` dataset from Wooldridge, which records the sale price and characteristics of 88 houses in the Boston area. The question is a familiar one in applied work. How much is an extra square foot, an extra bedroom, or a bigger lot worth to a buyer, holding the other features of the house constant?

By the end of this lab you should be able to do the following.

1. Load a Wooldridge dataset and estimate a multiple regression with `lm()`.
2. Present the results in a regression table.
3. Store coefficients, standard errors, R squared and other estimates in objects and report them inside your text.
4. Run two tailed and one tailed t tests by hand and with a command.
5. Run an F test of joint exclusion restrictions by hand and with a command.
6. Run an LM test by hand and with a command.
7. Create log variables and estimate a model in logs.
8. Draw the graphs that are useful after a regression.
9. Draw plots that help you detect heteroskedasticity.

Packages and data

We need five packages. If any of them is missing on your computer, install it once with `install.packages()` and then load it with `library()`.

```
install.packages(c("wooldridge", "stargazer", "car", "lmtest", "ggplot2"))
```

```
library(wooldridge)
library(stargazer)
library(car)
library(lmtest)
library(ggplot2)
```

The `wooldridge` package carries every dataset used in the textbook. The `data()` command loads one of them into the Environment.

```
data("hprice1")
str(hprice1)
```

```
## 'data.frame':   88 obs. of  10 variables:
## $ price      : num  300 370 191 195 373 ...
## $ assess     : num  349 352 218 232 319 ...
## $ bdrms      : int   4 3 3 3 4 5 3 3 3 3 ...
## $ lotsize    : num  6126 9903 5200 4600 6095 ...
## $ sqrft      : int  2438 2076 1374 1448 2514 2754 2067 1731 1767 1890 ...
## $ colonial   : int   1 1 0 1 1 1 1 1 0 0 ...
## $ lprice     : num   5.7 5.91 5.25 5.27 5.92 ...
## $ lassess    : num   5.86 5.86 5.38 5.45 5.77 ...
## $ llotsize   : num   8.72 9.2 8.56 8.43 8.72 ...
## $ lsqrft     : num   7.8 7.64 7.23 7.28 7.83 ...
## - attr(*, "time.stamp")= chr "25 Jun 2011 23:03"
```

We will use five variables. `price` is the sale price in thousands of dollars, `lotsize` is the size of the lot in

square feet, `sqrft` is the size of the house in square feet, `bdrms` is the number of bedrooms, and `colonial` is a dummy equal to one if the house is built in the colonial style. You can read the full description by typing `?hprice1` in the Console.

```
summary(hprice1[, c("price", "lotsize", "sqrft", "bdrms", "colonial")])
```

```
##      price      lotsize      sqrft      bdrms
##  Min.   :111.0   Min.    : 1000   Min.   :1171   Min.   :2.000
##  1st Qu.:230.0   1st Qu.: 5733   1st Qu.:1660   1st Qu.:3.000
##  Median :265.5   Median : 6430   Median :1845   Median :3.000
##  Mean   :293.5   Mean    : 9020   Mean    :2014   Mean    :3.568
##  3rd Qu.:326.2   3rd Qu.: 8583   3rd Qu.:2227   3rd Qu.:4.000
##  Max.   :725.0   Max.    :92681   Max.    :3880   Max.    :7.000
##      colonial
##  Min.    :0.0000
##  1st Qu.:0.0000
##  Median :1.0000
##  Mean    :0.6932
##  3rd Qu.:1.0000
##  Max.    :1.0000
```

Estimating a multiple regression

The model is

$$price = \beta_0 + \beta_1 lotsize + \beta_2 sqrft + \beta_3 bdrms + \beta_4 colonial + u$$

The syntax of `lm()` is the same as in simple regression. We just add the extra regressors with a plus sign.

```
model_level <- lm(price ~ lotsize + sqrft + bdrms + colonial, data = hprice1)
summary(model_level)
```

```
##
## Call:
## lm(formula = price ~ lotsize + sqrft + bdrms + colonial, data = hprice1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -122.268  -38.271   -6.545    28.210   218.040
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.413e+01  2.960e+01  -0.815   0.41741
## lotsize      2.076e-03  6.427e-04   3.230   0.00177 **
## sqrft       1.242e-01  1.334e-02   9.314  1.53e-14 ***
## bdrms       1.100e+01  9.515e+00   1.156   0.25080
## colonial    1.372e+01  1.464e+01   0.937   0.35146
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 59.88 on 83 degrees of freedom
## Multiple R-squared:  0.6758, Adjusted R-squared:  0.6602
## F-statistic: 43.25 on 4 and 83 DF,  p-value: < 2.2e-16
```

Each slope is a partial effect. The coefficient on `sqrft` tells us how the predicted price changes when the house is one square foot larger, holding lot size, bedrooms and style constant.

A regression table

The output of `summary()` is fine for our own eyes but it is not how results appear in a paper. The `stargazer()` function turns one or more models into a publication style table with standard errors in parentheses under each coefficient.

When you work in the Console or in an R script, use `type = "text"` to see the table on screen.

```
stargazer(model_level, type = "text")

##
## =====
##                               Dependent variable:
##                               -----
##                               price
## -----
## lotsize                0.002***
##                        (0.001)
##
## sqrft                   0.124***
##                        (0.013)
##
## bdrms                   11.004
##                        (9.515)
##
## colonial                13.716
##                        (14.637)
##
## Constant               -24.127
##                        (29.603)
##
## -----
## Observations              88
## R2                       0.676
## Adjusted R2              0.660
## Residual Std. Error    59.877 (df = 83)
## F Statistic            43.252*** (df = 4; 83)
## =====
## Note:                    *p<0.1; **p<0.05; ***p<0.01
```

Inside an R Markdown document we want a properly typeset table. For that we set `type = "latex"` when knitting to PDF or `type = "html"` when knitting to HTML, and we add `results = 'asis'` to the chunk header so that the table is printed as it is. The line below picks the right type for us.

```
table_type <- ifelse(knitr::is_latex_output(), "latex", "html")

stargazer(model_level,
  type = table_type,
  header = FALSE,
  title = "Determinants of house prices",
  label = "tab:level",
  dep.var.labels = "Price in thousands of dollars",
  covariate.labels = c("Lot size", "Square feet", "Bedrooms", "Colonial"),
```

```
digits = 4,
omit.stat = "ser")
```

Table 1: Determinants of house prices

	<i>Dependent variable:</i>
	Price in thousands of dollars
Lot size	0.0021*** (0.0006)
Square feet	0.1242*** (0.0133)
Bedrooms	11.0043 (9.5153)
Colonial	13.7155 (14.6373)
Constant	-24.1265 (29.6035)
Observations	88
R ²	0.6758
Adjusted R ²	0.6602
F Statistic	43.2521*** (df = 4; 83)
<i>Note:</i> *p<0.1; **p<0.05; ***p<0.01	

Storing estimates and using them in text

Typing numbers from the output into your text by hand invites mistakes, and the numbers go stale the moment you change the model. A better habit is to store every estimate in an object and let R write the number into the sentence.

Everything we need lives inside the model object and its summary.

```
sum_level <- summary(model_level)

b <- coef(model_level)
se <- sqrt(diag(vcov(model_level)))

b

## (Intercept)      lotsize      sqrft      bdrms      colonial
## -24.126528273  0.002075832  0.124237475  11.004292202  13.715542139
se

## (Intercept)      lotsize      sqrft      bdrms      colonial
## 2.960345e+01  6.426511e-04  1.333826e-02  9.515260e+00  1.463727e+01
```

Both `b` and `se` are named vectors, so we can pull out any element by the name of its variable.

```

b_sqrft <- b["sqrft"]
se_sqrft <- se["sqrft"]
b_bdrms <- b["bdrms"]
se_bdrms <- se["bdrms"]

r2 <- sum_level$r.squared
adj_r2 <- sum_level$adj.r.squared
sigma_hat <- sum_level$sigma

n <- nobs(model_level)
k <- length(b) - 1
df_ur <- df.residual(model_level)
ssr_ur <- sum(resid(model_level)^2)

c(n = n, k = k, df = df_ur, r2 = r2, adj_r2 = adj_r2, sigma_hat = sigma_hat, ssr = ssr_ur)

##           n           k           df           r2           adj_r2           sigma_hat
## 8.800000e+01 4.000000e+00 8.300000e+01 6.757919e-01 6.601674e-01 5.987697e+01
##           ssr
## 2.975759e+05

```

To place a stored value inside a sentence in R Markdown, write a backtick, the letter r, a space, the R expression, and a closing backtick. For example, typing

The coefficient on `sqrft` is ``r round(b_sqrft, 3)``.

produces the sentence “The coefficient on `sqrft` is 0.124.” when the document is knitted. The paragraph below was written entirely this way, so not a single number in it was typed by hand.

The sample has 88 houses and the model has 4 regressors, which leaves 83 degrees of freedom. An extra square foot raises the predicted price by 124 dollars, holding the other regressors constant, with a standard error of 13 dollars. An extra bedroom raises the predicted price by 11 thousand dollars with a standard error of 9.52. The model explains 67.6 percent of the variation in price, and the adjusted R squared is 0.66. The standard error of the regression is 59.88 thousand dollars.

The t test

Two tailed test

We start with the test that `summary()` reports by default. For the bedrooms coefficient,

$$H_0: \beta_3 = 0 \quad \text{against} \quad H_1: \beta_3 \neq 0$$

The t statistic is the estimate minus the hypothesized value, divided by the standard error. We compare it with the critical value from a t distribution with $n - k - 1$ degrees of freedom, and we get the p value as the area in both tails beyond the statistic.

```

alpha <- 0.05

t_bdrms <- (b_bdrms - 0) / se_bdrms
crit_two <- qt(1 - alpha / 2, df = df_ur)
p_two <- 2 * pt(abs(t_bdrms), df = df_ur, lower.tail = FALSE)

c(t = unname(t_bdrms), critical_value = crit_two, p_value = unname(p_two))

```

```
##           t critical_value      p_value
##      1.1564888      1.9889598      0.2507991
```

The command version is already in the coefficient table of the summary. The columns `t value` and `Pr(>|t|)` match what we just computed.

```
coef(sum_level)
```

```
##           Estimate Std. Error   t value   Pr(>|t|)
## (Intercept) -24.126528273 2.960345e+01 -0.8149903 4.174103e-01
## lotsize      0.002075832 6.426511e-04  3.2301082 1.774189e-03
## sqrft        0.124237475 1.333826e-02  9.3143700 1.534380e-14
## bdrms        11.004292202 9.515260e+00  1.1564888 2.507991e-01
## colonial    13.715542139 1.463727e+01  0.9370290 3.514622e-01
```

```
coef(sum_level)["bdrms", ]
```

```
##      Estimate Std. Error   t value   Pr(>|t|)
## 11.0042922   9.5152604   1.1564888   0.2507991
```

The t statistic of 1.16 is smaller in absolute value than the critical value of 1.99, and the p value of 0.251 is above 0.05. We fail to reject the null at the 5 percent level. Once house size, lot size and style are held constant, the data cannot tell the effect of an extra bedroom apart from zero.

The same logic gives a 95 percent confidence interval, which we can build by hand or with `confint()`.

```
c(lower = unname(b_bdrms - crit_two * se_bdrms), upper = unname(b_bdrms + crit_two * se_bdrms))
```

```
##      lower      upper
## -7.921178 29.929762
```

```
confint(model_level, "bdrms", level = 0.95)
```

```
##           2.5 %    97.5 %
## bdrms -7.921178 29.92976
```

One tailed test

Economic reasoning often tells us the sign of an effect in advance. Nobody expects an extra bedroom to lower the price of a house, so a one sided alternative is sensible.

$$H_0: \beta_3 = 0 \quad \text{against} \quad H_1: \beta_3 > 0$$

The t statistic does not change. What changes is the rejection rule. All of the 5 percent now sits in the right tail, so the critical value is smaller and the p value is the area to the right of the statistic only.

```
crit_one <- qt(1 - alpha, df = df_ur)
p_one <- pt(t_bdrms, df = df_ur, lower.tail = FALSE)
```

```
c(t = unname(t_bdrms), critical_value = crit_one, p_value = unname(p_one))
```

```
##           t critical_value      p_value
##      1.1564888      1.6634202      0.1253996
```

R has no separate command for a one tailed test in a regression, and it does not need one. When the estimate has the sign predicted by the alternative, the one tailed p value is half of the two tailed p value from the summary.

```
coef(sum_level)["bdrms", "Pr(>|t|)"] / 2
```

```
## [1] 0.1253996
```

The one tailed p value is 0.125 and the critical value is 1.66. The p value is exactly half of the two tailed one, so a one sided alternative always makes it easier to reject. Here the t statistic of 1.16 still falls short of the critical value, and we fail to reject the null at the 5 percent level. Because the one tailed test is the more lenient of the two, the choice of alternative must come from economic reasoning made before looking at the estimates, not from a wish to see a star in the table.

Testing a value other than zero

The null does not have to be zero. Suppose a real estate agent claims that every extra square foot adds 100 dollars to the price. Since price is measured in thousands, the claim is

$$H_0: \beta_2 = 0.10 \quad \text{against} \quad H_1: \beta_2 \neq 0.10$$

```
t_sqrft <- (b_sqrft - 0.10) / se_sqrft
p_sqrft <- 2 * pt(abs(t_sqrft), df = df_ur, lower.tail = FALSE)

c(t = unname(t_sqrft), p_value = unname(p_sqrft))
```

```
##           t    p_value
## 1.8171394 0.0728049
```

The `linearHypothesis()` function from the `car` package runs this test with one line. It reports an F statistic, which for a single restriction is the square of the t statistic, so the p value is identical.

```
linearHypothesis(model_level, "sqrft = 0.10")
```

```
## Linear hypothesis test
##
## Hypothesis:
##   sqrft = 0.1
##
## Model 1: restricted model
## Model 2: price ~ lotsize + sqrft + bdrms + colonial
##
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      84 309414
## 2      83 297576   1    11838 3.302 0.0728 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

unname(t_sqrft^2)

## [1] 3.301996
```

The F test

A t test handles one restriction. To test several at once we need the F test. Bedrooms and colonial style both look weak on their own, so we ask whether they matter jointly.

$$H_0: \beta_3 = 0 \text{ and } \beta_4 = 0 \quad \text{against} \quad H_1: H_0 \text{ is not true}$$

By hand

The unrestricted model is the one we already estimated. The restricted model imposes the null by dropping the two variables. The F statistic compares how much the sum of squared residuals rises when we impose the restrictions.

$$F = \frac{(SSR_r - SSR_{ur})/q}{SSR_{ur}/(n - k - 1)}$$

```
model_restricted <- lm(price ~ lotsize + sqrft, data = hprice1)

ssr_r <- sum(resid(model_restricted)^2)
q <- 2

F_stat <- ((ssr_r - ssr_ur) / q) / (ssr_ur / df_ur)
F_crit <- qf(1 - alpha, df1 = q, df2 = df_ur)
F_p <- pf(F_stat, df1 = q, df2 = df_ur, lower.tail = FALSE)

c(F = F_stat, critical_value = F_crit, p_value = F_p)

##              F critical_value      p_value
##      1.6191502      3.1065071      0.2042598
```

Because both models have the same dependent variable, the R squared form of the statistic gives exactly the same number.

$$F = \frac{(R_{ur}^2 - R_r^2)/q}{(1 - R_{ur}^2)/(n - k - 1)}$$

```
r2_r <- summary(model_restricted)$r.squared

((r2 - r2_r) / q) / ((1 - r2) / df_ur)

## [1] 1.61915
```

By command

There are two convenient commands. `anova()` takes the restricted and the unrestricted model, and `linearHypothesis()` takes the unrestricted model and the restrictions written out as text.

```
anova(model_restricted, model_level)

## Analysis of Variance Table
##
## Model 1: price ~ lotsize + sqrft
## Model 2: price ~ lotsize + sqrft + bdrms + colonial
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      85 309186
## 2      83 297576  2     11610 1.6192 0.2043

linearHypothesis(model_level, c("bdrms = 0", "colonial = 0"))

## Linear hypothesis test
##
## Hypothesis:
## bdrms = 0
## colonial = 0
```

```
##
## Model 1: restricted model
## Model 2: price ~ lotsize + sqrft + bdrms + colonial
##
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      85 309186
## 2      83 297576  2    11610 1.6192 0.2043
```

The F statistic is 1.62 against a 5 percent critical value of 3.11, with a p value of 0.204. We fail to reject the null that bedrooms and colonial style are jointly irrelevant at the 5 percent level.

Overall significance of the regression

The F statistic printed at the bottom of `summary()` tests the null that all slopes are zero. The restricted model then contains only an intercept, its R squared is zero, and the formula simplifies.

```
F_overall <- (r2 / k) / ((1 - r2) / df_ur)
F_overall
```

```
## [1] 43.2521
```

```
sum_level$fstatistic
```

```
##   value  numdf  dendif
## 43.2521  4.0000 83.0000
```

The LM test

The F test relies on normally distributed errors for its exact distribution. The Lagrange multiplier test, also called the nR^2 test, relies only on a large sample. It needs only the restricted model, and it follows a chi square distribution with q degrees of freedom under the null.

By hand

We test the same null as before, that bedrooms and colonial style can be dropped. The steps are as follows.

1. Estimate the restricted model and save its residuals, $\tilde{u}$.
2. Regress $\tilde{u}$ on all of the independent variables, including the ones excluded under the null, and save the R squared, R_u^2 .
3. Compute $LM = nR_u^2$ and compare it with a χ_q^2 critical value.

```
hprice1$u_tilde <- resid(model_restricted)

aux_lm <- lm(u_tilde ~ lotsize + sqrft + bdrms + colonial, data = hprice1)
r2_u <- summary(aux_lm)$r.squared

LM_stat <- n * r2_u
LM_crit <- qchisq(1 - alpha, df = q)
LM_p <- pchisq(LM_stat, df = q, lower.tail = FALSE)

c(LM = LM_stat, critical_value = LM_crit, p_value = LM_p)
```

```
##           LM critical_value      p_value
##      3.3044533      5.9914645      0.1916228
```

The intuition is simple. If bedrooms and colonial style truly do not belong in the model, the restricted residuals should be unrelated to them and the auxiliary R squared should be close to zero.

By command

Base R has no ready command for the LM test of exclusion restrictions, so we write our own. A function is just the steps above wrapped up so that they can be reused with any pair of models.

```
lm_test <- function(restricted, unrestricted) {  
  data_used <- model.frame(unrestricted)  
  data_used$u_tilde <- resid(restricted)  
  aux <- update(unrestricted, u_tilde ~ ., data = data_used)  
  q <- df.residual(restricted) - df.residual(unrestricted)  
  LM <- nobs(unrestricted) * summary(aux)$r.squared  
  c(LM = LM, df = q, p_value = pchisq(LM, df = q, lower.tail = FALSE))  
}
```

```
lm_test(model_restricted, model_level)
```

```
##          LM          df    p_value  
## 3.3044533 2.0000000 0.1916228
```

A close relative is available as a command. Adding `test = "Chisq"` to `linearHypothesis()` gives the large sample chi square version of the F test, which equals q times the F statistic. It is not the same number as the LM statistic, but the two are asymptotically equivalent and they will almost always lead to the same decision.

```
linearHypothesis(model_level, c("bdrms = 0", "colonial = 0"), test = "Chisq")
```

```
## Linear hypothesis test  
##  
## Hypothesis:  
## bdrms = 0  
## colonial = 0  
##  
## Model 1: restricted model  
## Model 2: price ~ lotsize + sqrft + bdrms + colonial  
##  
##   Res.Df    RSS Df Sum of Sq  Chisq Pr(>Chisq)  
## 1      85 309186  
## 2      83 297576  2    11610  3.2383    0.1981
```

The LM statistic is 3.3 with a p value of 0.192, against the F test p value of 0.204. The two tests agree.

Creating log variables

House prices and house sizes are skewed to the right, and economists usually think about them in percentage terms. Both are good reasons to take logs. We create the new variables with `log()`, which in R is the natural log.

```
hprice1$log_price <- log(hprice1$price)  
hprice1$log_lotsize <- log(hprice1$lotsize)  
hprice1$log_sqrft <- log(hprice1$sqrft)  
  
head(hprice1[, c("price", "log_price", "lotsize", "log_lotsize", "sqrft", "log_sqrft")])
```

```
##      price log_price lotsize log_lotsize  sqrft log_sqrft  
## 1 300.000  5.703782   6126   8.720297  2438  7.798933  
## 2 370.000  5.913503   9903   9.200593  2076  7.638198  
## 3 191.000  5.252273   5200   8.556414  1374  7.225481  
## 4 195.000  5.273000   4600   8.433812  1448  7.277939
```

```
## 5 373.000  5.921578    6095    8.715224  2514  7.829630
## 6 466.275  6.144776    8566    9.055556  2754  7.920810
```

We leave `bdrms` and `colonial` in levels. Bedrooms is a small count and colonial is a dummy, and the log of zero is not defined.

$$\log(\text{price}) = \beta_0 + \beta_1 \log(\text{lotsize}) + \beta_2 \log(\text{sqrft}) + \beta_3 \text{bdrms} + \beta_4 \text{colonial} + u$$

```
model_log <- lm(log_price ~ log_lotsize + log_sqrft + bdrms + colonial, data = hprice1)
summary(model_log)
```

```
##
## Call:
## lm(formula = log_price ~ log_lotsize + log_sqrft + bdrms + colonial,
##     data = hprice1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.69479 -0.09750 -0.01619  0.09151  0.70228
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.34959    0.65104  -2.073   0.0413 *
## log_lotsize  0.16782    0.03818   4.395 3.25e-05 ***
## log_sqrft    0.70719    0.09280   7.620 3.69e-11 ***
## bdrms        0.02683    0.02872   0.934   0.3530
## colonial     0.05380    0.04477   1.202   0.2330
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1841 on 83 degrees of freedom
## Multiple R-squared:  0.6491, Adjusted R-squared:  0.6322
## F-statistic: 38.38 on 4 and 83 DF,  p-value: < 2.2e-16
```

You can also take logs inside the formula, as in `lm(log(price) ~ log(lotsize) + log(sqrft) + bdrms + colonial, data = hprice1)`. The estimates are identical.

Putting the two models side by side in one table makes them easy to compare.

```
stargazer(model_level, model_log,
  type = table_type,
  header = FALSE,
  title = "House prices in levels and in logs",
  label = "tab:both",
  column.labels = c("Level", "Log"),
  dep.var.labels = c("Price", "log(Price)"),
  digits = 4,
  omit.stat = "ser")
```

```
b_log <- coef(model_log)
se_log <- sqrt(diag(vcov(model_log)))
r2_log <- summary(model_log)$r.squared
```

The interpretation changes with the functional form. The coefficients on the logged regressors are elasticities. A 1 percent increase in house size raises the predicted price by about 0.71 percent, and a 1 percent increase in lot size raises it by about 0.17 percent, holding the other regressors constant. The coefficient on bedrooms

Table 2: House prices in levels and in logs

	<i>Dependent variable:</i>	
	Price Level	log(Price) Log
	(1)	(2)
lotsize	0.0021*** (0.0006)	
sqrft	0.1242*** (0.0133)	
log_lotsize		0.1678*** (0.0382)
log_sqrft		0.7072*** (0.0928)
bdrms	11.0043 (9.5153)	0.0268 (0.0287)
colonial	13.7155 (14.6373)	0.0538 (0.0448)
Constant	-24.1265 (29.6035)	-1.3496** (0.6510)
Observations	88	88
R ²	0.6758	0.6491
Adjusted R ²	0.6602	0.6322
F Statistic (df = 4; 83)	43.2521***	38.3785***
<i>Note:</i>	*p<0.1; **p<0.05; ***p<0.01	

is a semi elasticity, so one more bedroom raises the predicted price by about 2.7 percent. Do not compare the R squared of 0.649 with the 0.676 from the level model, since the two dependent variables are different.

The log model also invites a natural test. Is the elasticity of price with respect to house size equal to one, so that a house that is 10 percent bigger sells for 10 percent more?

```
t_elast <- (b_log["log_sqrft"] - 1) / se_log["log_sqrft"]
p_elast <- 2 * pt(abs(t_elast), df = df.residual(model_log), lower.tail = FALSE)

c(t = unname(t_elast), p_value = unname(p_elast))
```

```
##           t           p_value
## -3.155179317 0.002234697
```

```
linearHypothesis(model_log, "log_sqrft = 1")
```

```
## Linear hypothesis test
##
## Hypothesis:
## log_sqrft = 1
##
## Model 1: restricted model
## Model 2: log_price ~ log_lotsize + log_sqrft + bdrms + colonial
##
##   Res.Df    RSS Df Sum of Sq    F   Pr(>F)
## 1      84 3.1511
## 2      83 2.8136  1   0.33747 9.9552 0.002235 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Graphs after a regression

A table tells you what the estimates are. Graphs tell you whether to believe them. We first collect the fitted values and residuals from both models in the data frame.

```
hprice1$fitted_level <- fitted(model_level)
hprice1$resid_level <- resid(model_level)
hprice1$fitted_log <- fitted(model_log)
hprice1$resid_log <- resid(model_log)
```

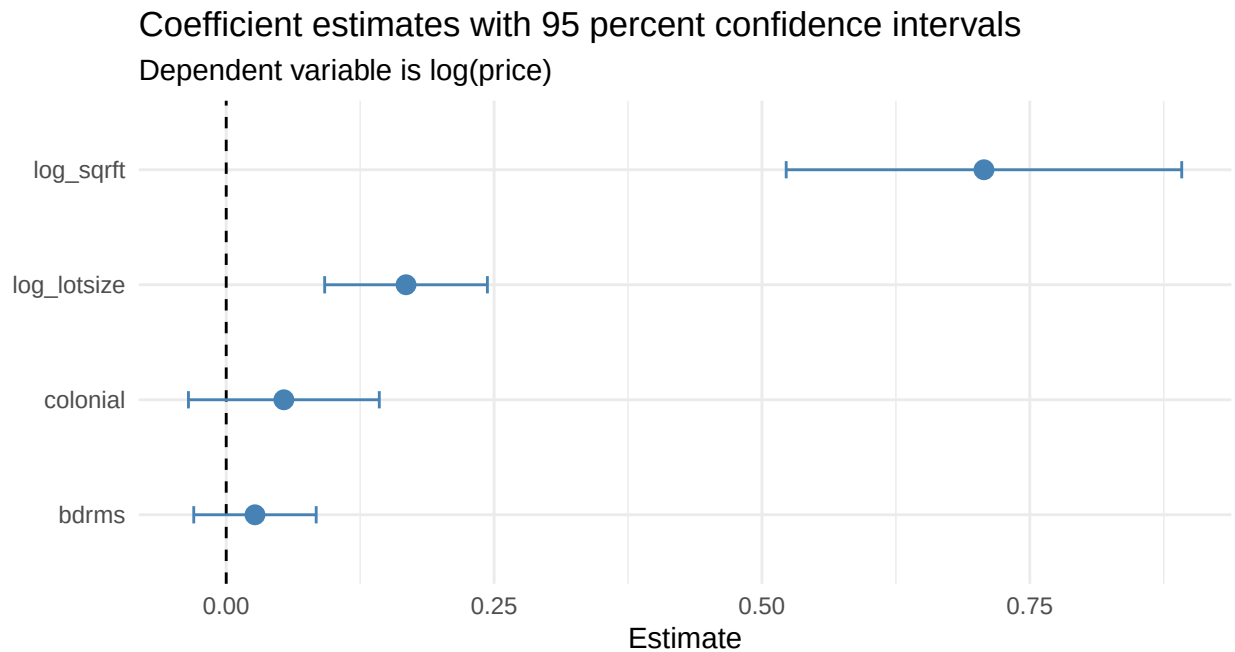
Coefficient plot

A coefficient plot shows each estimate with its 95 percent confidence interval. An interval that crosses zero means the coefficient is not significant at the 5 percent level in a two tailed test. We use the log model because its coefficients are on comparable scales.

```
coef_data <- data.frame(
  term = names(b_log),
  estimate = b_log,
  lower = confint(model_log)[, 1],
  upper = confint(model_log)[, 2]
)
coef_data <- coef_data[coef_data$term != "(Intercept)", ]

ggplot(coef_data, aes(x = estimate, y = term)) +
  geom_vline(xintercept = 0, linetype = "dashed") +
```

```
geom_errorbar(aes(xmin = lower, xmax = upper), width = 0.15, orientation = "y", color = "steelblue") +
geom_point(size = 3, color = "steelblue") +
labs(
  title = "Coefficient estimates with 95 percent confidence intervals",
  subtitle = "Dependent variable is log(price)",
  x = "Estimate",
  y = NULL
) +
theme_minimal()
```



Actual against fitted values

If the model fits well, the points gather around the 45 degree line. Systematic departures from the line point to a problem with the functional form.

```
ggplot(hprice1, aes(x = fitted_level, y = price)) +
  geom_point(color = "steelblue", alpha = 0.7) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed") +
  labs(
    title = "Actual against fitted house prices",
    subtitle = "Level model, dashed line is the 45 degree line",
    x = "Fitted price in thousands of dollars",
    y = "Actual price in thousands of dollars"
  ) +
  theme_minimal()
```

Actual against fitted house prices
Level model, dashed line is the 45 degree line



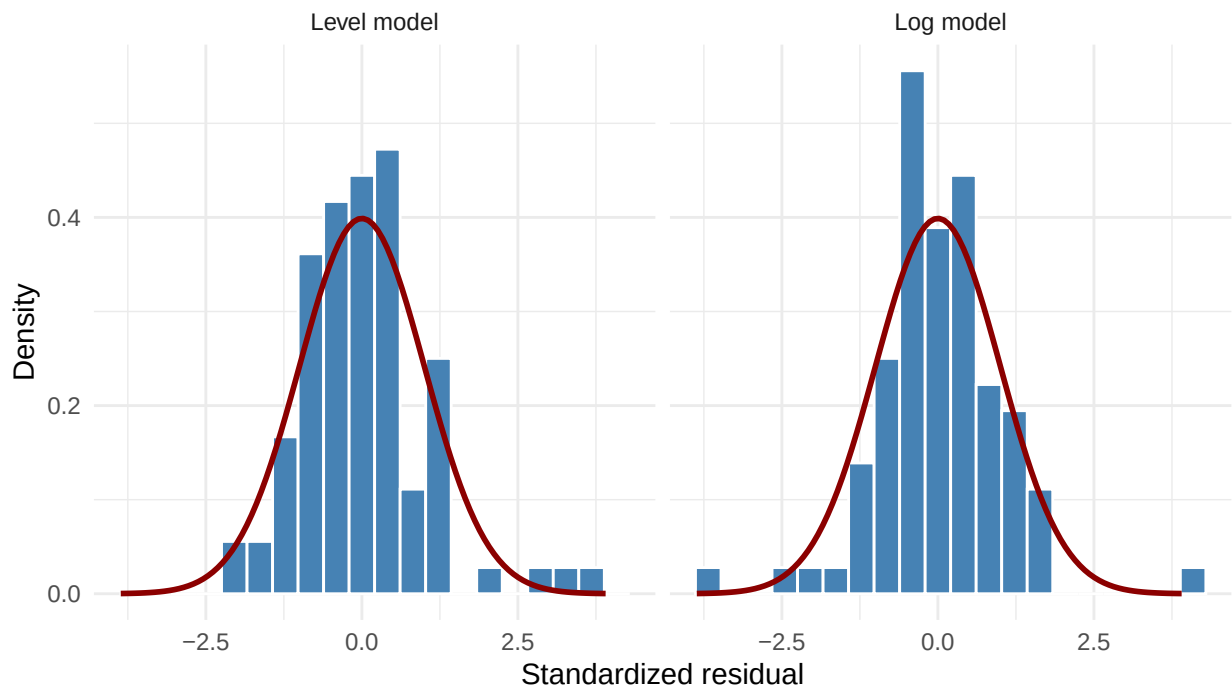
Distribution of the residuals

The exact t and F tests assume normal errors. A histogram and a normal Q-Q plot of the residuals show how reasonable that is. In the Q-Q plot, normal residuals fall along the straight line.

```
resid_data <- data.frame(
  residual = c(scale(hprice1$resid_level), scale(hprice1$resid_log)),
  model = rep(c("Level model", "Log model"), each = n)
)

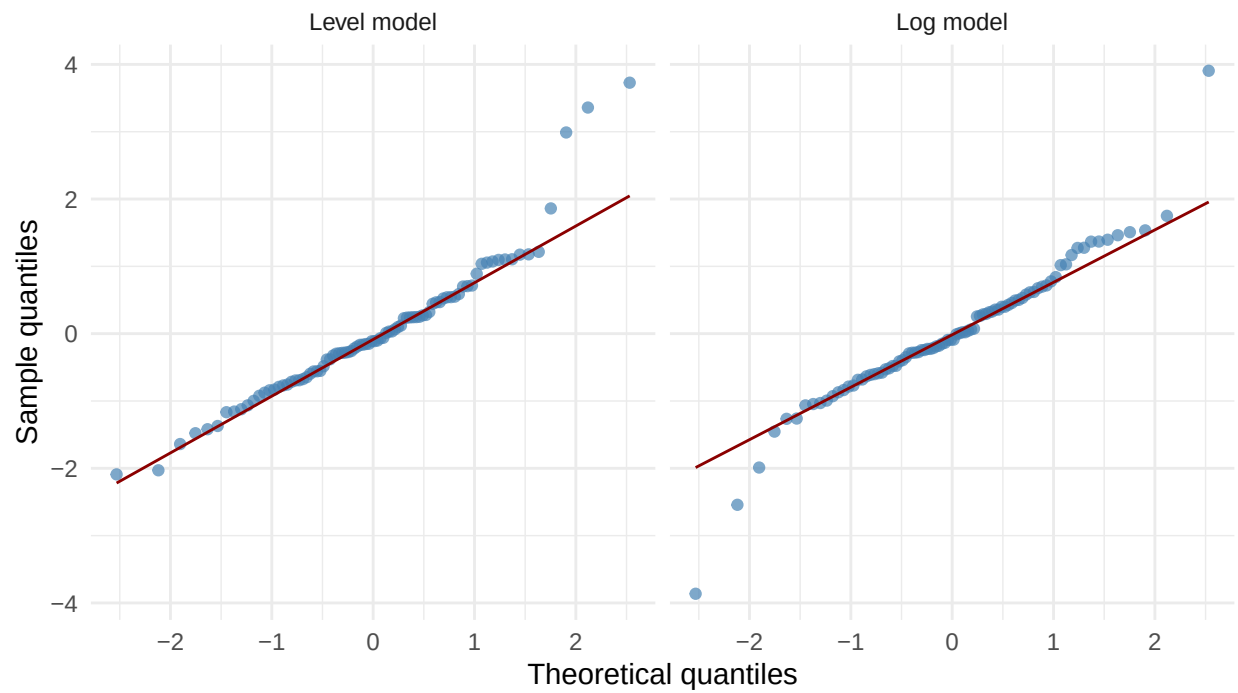
ggplot(resid_data, aes(x = residual)) +
  geom_histogram(aes(y = after_stat(density)), bins = 20, fill = "steelblue", color = "white") +
  stat_function(fun = dnorm, color = "darkred", linewidth = 1) +
  facet_wrap(~ model) +
  labs(
    title = "Standardized residuals against a standard normal density",
    x = "Standardized residual",
    y = "Density"
  ) +
  theme_minimal()
```

Standardized residuals against a standard normal density



```
ggplot(resid_data, aes(sample = residual)) +  
  stat_qq(color = "steelblue", alpha = 0.7) +  
  stat_qq_line(color = "darkred") +  
  facet_wrap(~ model) +  
  labs(  
    title = "Normal Q-Q plots of the residuals",  
    x = "Theoretical quantiles",  
    y = "Sample quantiles"  
  ) +  
  theme_minimal()
```

Normal Q-Q plots of the residuals

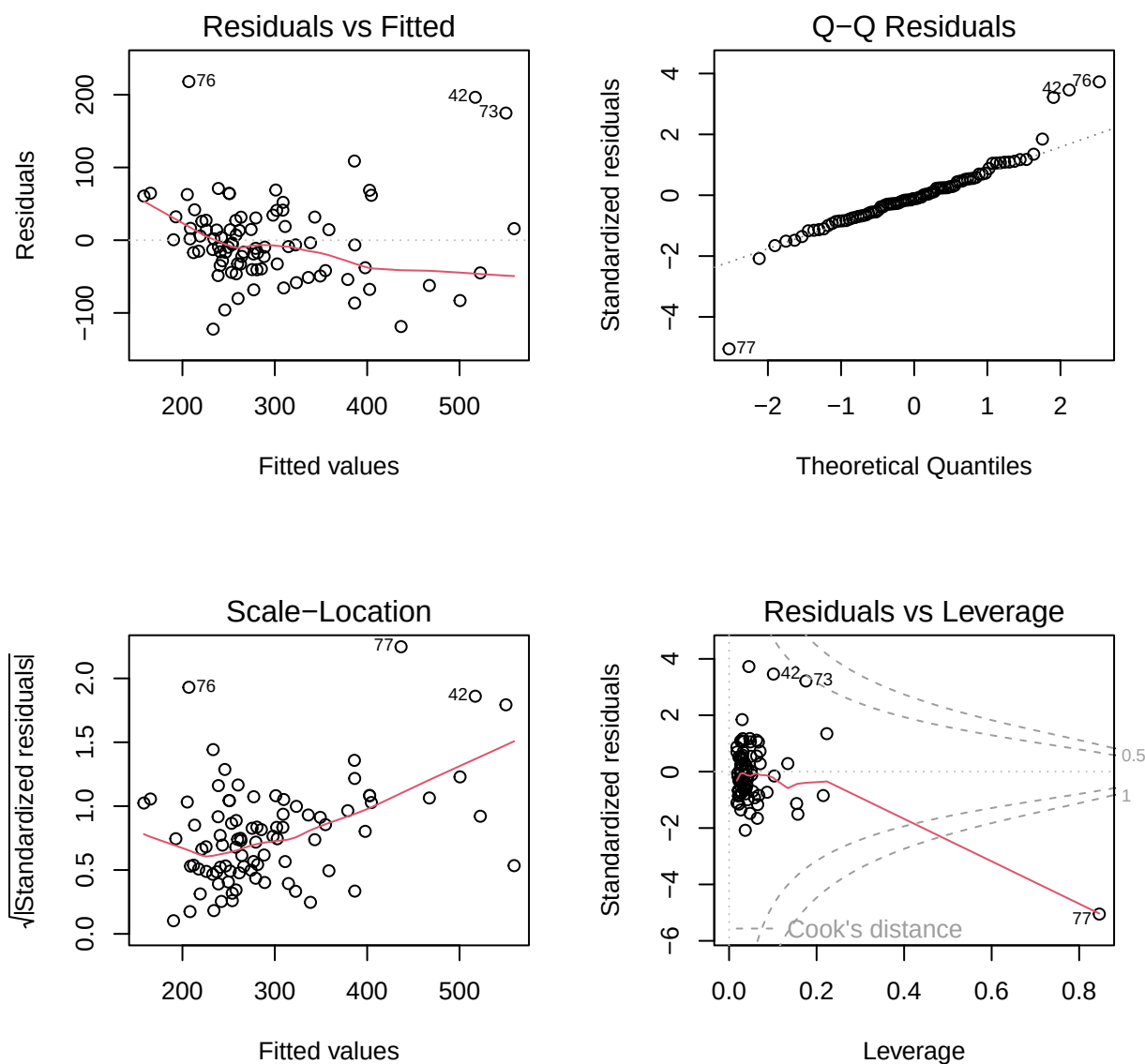


The level model has a long right tail driven by a few very expensive houses. Taking logs pulls that tail in.

The built in diagnostic plots

Base R produces four standard diagnostic plots from any model object with a single call to `plot()`.

```
par(mfrow = c(2, 2))  
plot(model_level)
```



```
par(mfrow = c(1, 1))
```

Plots to detect heteroskedasticity

Homoskedasticity says that the variance of the error is the same at every value of the regressors. If that holds, a plot of the residuals against the fitted values looks like a band of roughly constant width. A funnel that opens up as the fitted values grow is the classic sign of heteroskedasticity.

Residuals against fitted values

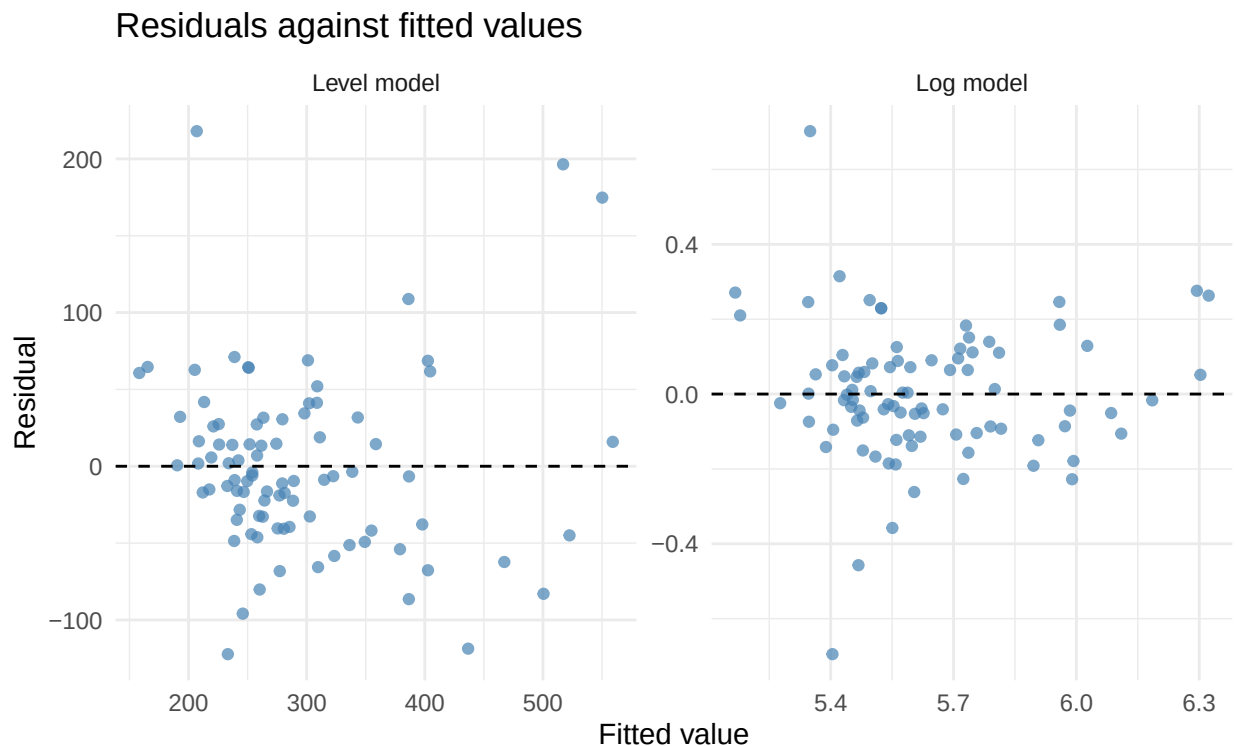
```
het_data <- data.frame(
  fitted = c(hprice1$fitted_level, hprice1$fitted_log),
  residual = c(hprice1$resid_level, hprice1$resid_log),
```

```

model = rep(c("Level model", "Log model"), each = n)
)

ggplot(het_data, aes(x = fitted, y = residual)) +
  geom_point(color = "steelblue", alpha = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  facet_wrap(~ model, scales = "free") +
  labs(
    title = "Residuals against fitted values",
    x = "Fitted value",
    y = "Residual"
  ) +
  theme_minimal()

```



Squared residuals against fitted values

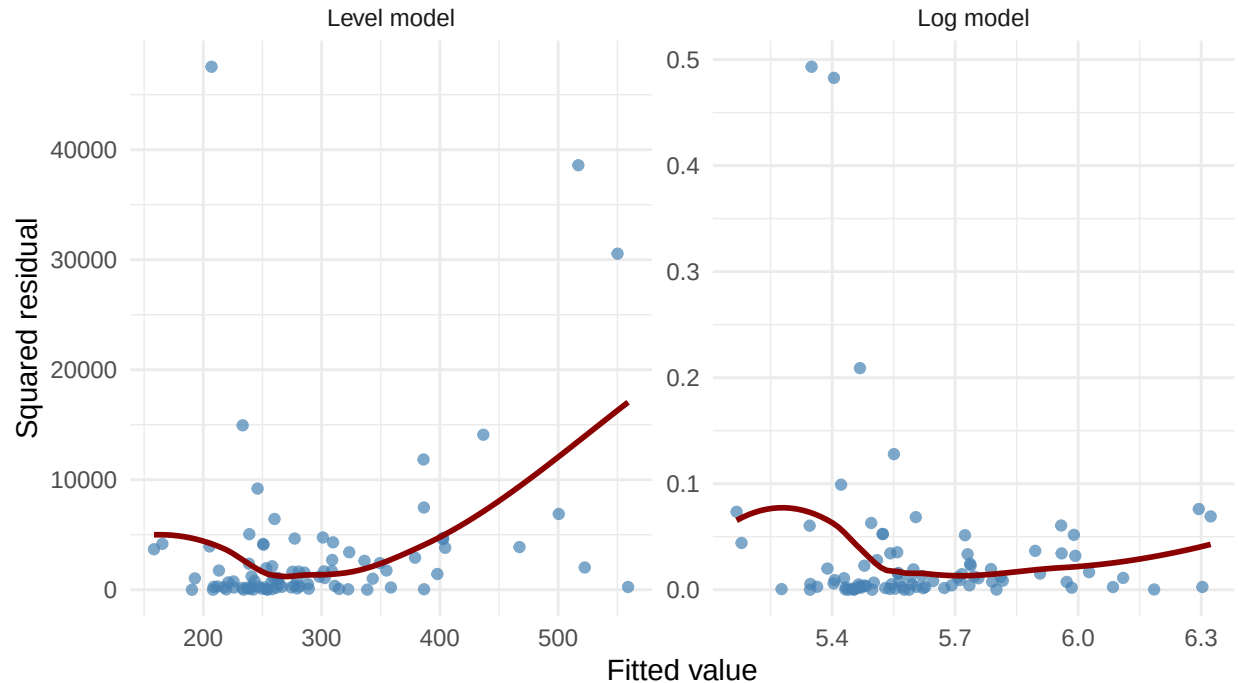
The squared residual is our estimate of the error variance for each house, so plotting it shows the variance directly. Under homoskedasticity the smooth line should be flat.

```

ggplot(het_data, aes(x = fitted, y = residual^2)) +
  geom_point(color = "steelblue", alpha = 0.7) +
  geom_smooth(method = "loess", se = FALSE, color = "darkred") +
  facet_wrap(~ model, scales = "free") +
  labs(
    title = "Squared residuals against fitted values",
    x = "Fitted value",
    y = "Squared residual"
  ) +
  theme_minimal()

```

Squared residuals against fitted values



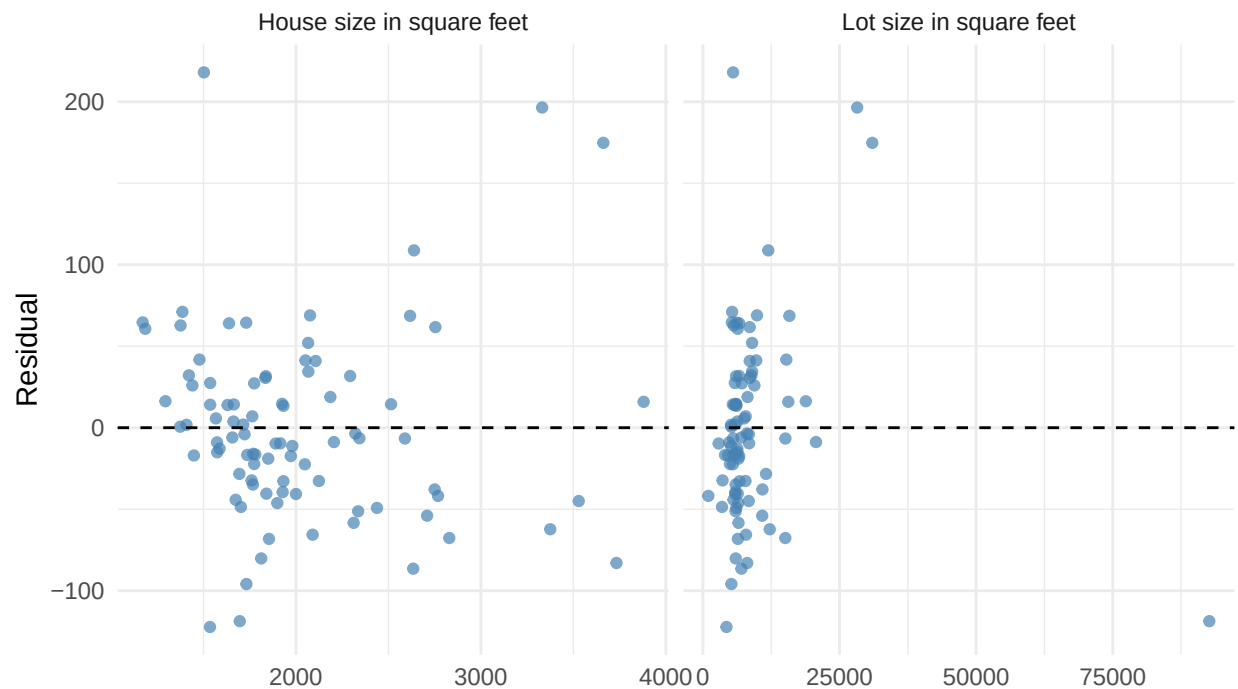
Residuals against each regressor

Heteroskedasticity is often tied to one particular variable. Plotting the residuals against each regressor in turn helps to find the source.

```
reg_data <- data.frame(
  value = c(hprice1$sqrft, hprice1$lotsize),
  residual = rep(hprice1$resid_level, 2),
  regressor = rep(c("House size in square feet", "Lot size in square feet"), each = n)
)

ggplot(reg_data, aes(x = value, y = residual)) +
  geom_point(color = "steelblue", alpha = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  facet_wrap(~ regressor, scales = "free_x") +
  labs(
    title = "Level model residuals against the regressors",
    x = NULL,
    y = "Residual"
  ) +
  theme_minimal()
```

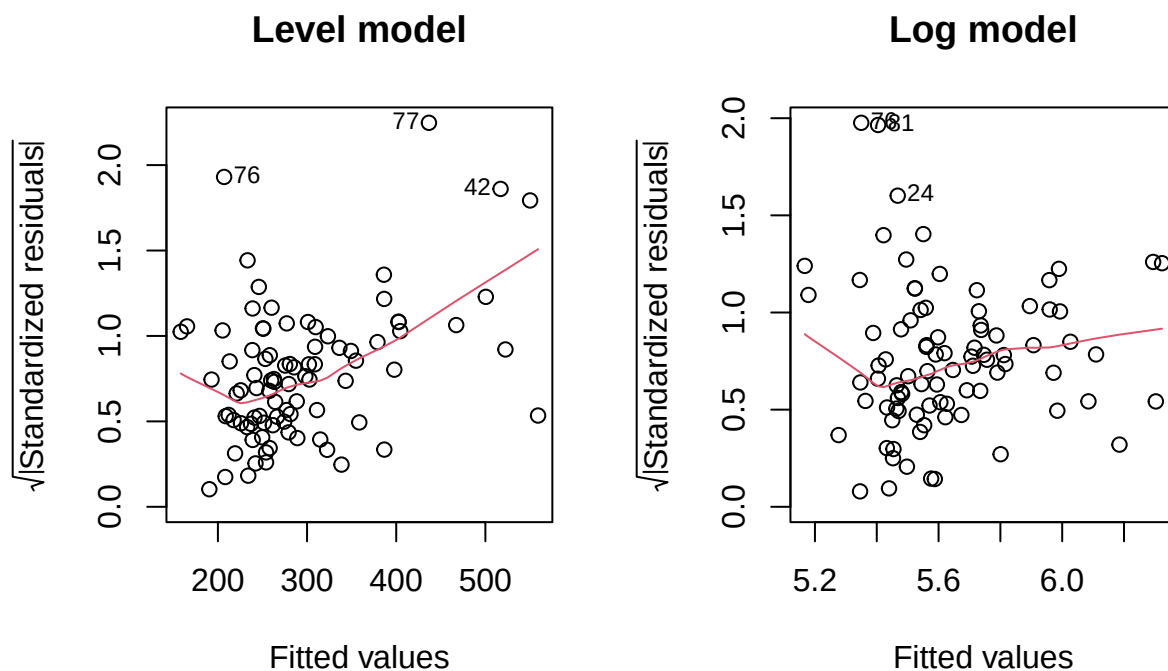
Level model residuals against the regressors



Scale location plot

The scale location plot puts the square root of the absolute standardized residual on the vertical axis. It is the third of the four base R diagnostic plots, and an upward sloping line is evidence that the spread of the residuals grows with the fitted values.

```
par(mfrow = c(1, 2))
plot(model_level, which = 3, caption = "", main = "Level model")
plot(model_log, which = 3, caption = "", main = "Log model")
```



```
par(mfrow = c(1, 1))
```

Confirming the picture with a test

A plot suggests and a test confirms. The Breusch Pagan test has the null that the error variance is constant, and `bptest()` from the `lmtest` package runs it in one line. We apply it to both models.

```
bp_level <- bptest(model_level)
bp_log <- bptest(model_log)

bp_level

##
## studentized Breusch-Pagan test
##
## data: model_level
## BP = 16.151, df = 4, p-value = 0.002824

bp_log

##
## studentized Breusch-Pagan test
##
## data: model_log
## BP = 5.9134, df = 4, p-value = 0.2057
```

The plots and the tests tell the same story. In the level model the residuals fan out as the fitted price rises, and the Breusch Pagan test rejects homoskedasticity with a p value of 0.0028. In the log model the spread is far more even and the p value is 0.206, so we can no longer reject constant variance at the 5 percent level. Taking logs is often the simplest remedy for heteroskedasticity, although it is not guaranteed to work in every dataset.

Heteroskedasticity does not bias the OLS coefficients, but it does make the usual standard errors, and therefore every t, F and LM statistic in this lab, unreliable for the level model. We will see how to correct the standard errors in a later lab.

Summary and questions to think about

We estimated a multiple regression on real data, reported it in a table, and stored the estimates in objects so that the text of the report updates itself. We tested a single coefficient with two tailed and one tailed t tests, tested two exclusion restrictions jointly with the F test and the LM test, and checked every result computed by hand against a command. We then built log variables, estimated a log model, and used graphs to judge the fit of the model, the normality of the residuals, and the presence of heteroskedasticity.

Before you submit this lab, work through the following questions.

1. Load the `wage1` dataset and regress `wage` on `educ`, `exper` and `tenure`. Present the results in a `stargazer` table and write a short paragraph that reports the coefficient on `educ`, its standard error and the R squared using inline R code only.
2. In that model, test whether the return to a year of education is zero against the alternative that it is positive. Do the test by hand and confirm the p value from the summary output.
3. Test whether `exper` and `tenure` are jointly insignificant, using the F test by hand, `anova()`, and the LM test by hand. Do the three approaches agree?
4. Create `log_wage` and estimate the model again. How does the interpretation of the coefficient on `educ` change?
5. Plot the residuals against the fitted values for both wage models and run `bptest()` on each. Which model shows stronger evidence of heteroskedasticity, and why might that be?